

Reviewer Pack — Minimal Order of Automorphic Representations and Resolution ...

Entry c41170a41b57 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 16:01:30 UTC

Minimal Order of Automorphic Representations and Resolution Proof Width Correlation

Entry ID: c41170a41b57 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 16:01:30 UTC

1. Conjecture

Field A (mathematical branch): Automorphic Forms Theory **Field B** (complexity object): Complexity Theory (Resolution Proof Complexity)

Statement:

The minimal order of automorphic representations associated with a given polynomial f over the finite field F_q , denoted as $\min_order(f)$, is linearly correlated with its resolution proof width, such that $\min_order(f) = \Theta(w(\varphi_f))$, where φ_f is the Tseitin formula encoding f .

Rationale (proposer's reasoning):

Automorphic forms provide a rich source of algebraic structures that could potentially be exploited to understand the complexity of Boolean functions. The minimal order of automorphic representations reflects the arithmetic complexity of the function and might reveal non-trivial structural features that are reflected in its resolution proof width.

Taxonomy category: AutomorphicForms × ResolutionProofComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 014446ad8c787737

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all 30 random polynomial seeds, the correlation coefficient between $\min_order(f)$ and $w(\varphi_f)$ exceeds 0.7 and there is no seed where $\min_order(f) / w(\varphi_f) < 0.5$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.90	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
from itertools import combinations, product

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        max_row = i
        for j in range(i+1, m):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        pivot = A[i][i]
        for j in range(n):
            A[i][j] /= pivot
        for j in range(m):
            if j != i:
                factor = A[j][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]

def matrix_multiply(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
```

```

    return abs(a*b) // gcd(a, b)

def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

def generate_primes(k):
    primes = []
    num = 2
    while len(primes) < k:
        if is_prime(num):
            primes.append(num)
        num += 1
    return primes

def random_polynomial(n, q):
    return [random.randint(0, q-1) for _ in range(n+1)]

def tseitin_formula(poly):
    n = len(poly) - 1
    variables = list(range(2*n + 1))
    clauses = []

    for i in range(n+1):
        if poly[i] != 0:
            clause = [variables[2*i]]
            if poly[i] == -1:
                clause.append(-variables[2*i + 1])
            clauses.append(clause)

    for i in range(1, n+1):
        for j in range(i):
            clause = [-variables[2*j], variables[2*i]]
            clauses.append(clause)
            clause = [variables[2*j + 1], -variables[2*i + 1]]
            clauses.append(clause)

    return variables, clauses

def min_order(f, q):
    n = len(f) - 1
    # Simplified mapping for demonstration purposes
    return n * q

def resolution_width(phi_f):
    # Simplified mapping for demonstration purposes
    return len(phi_f)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    primes = generate_primes(30)
    q = primes[random.randint(0, 29)]

```

```

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    f = random_polynomial(n, q)
    variables, clauses = tseitin_formula(f)
    min_order_f = min_order(f, q)
    w_phi_f = resolution_width(clauses)

    results.append({
        "n": n,
        "min_order_f": min_order_f,
        "w_phi_f": w_phi_f
    })

correlation_sum = 0
ratio_sum = 0
instances_tested = len(results)
n_max = max(result["n"] for result in results)

for result in results:
    if result["min_order_f"] == 0 or result["w_phi_f"] == 0:
        continue
    correlation_sum += abs(result["min_order_f"] - result["w_phi_f"])
    ratio_sum += result["min_order_f"] / result["w_phi_f"]

if instances_tested == 0:
    return {
        "metric_name": "correlation",
        "metric_value": 0,
        "instances_tested": 0,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "no_valid_instances"
    }

correlation = correlation_sum / instances_tested
ratio_avg = ratio_sum / instances_tested

return {
    "metric_name": "correlation",
    "metric_value": correlation,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": correlation > 0.7 and ratio_avg >= 0.5,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or generate_primes(30)

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...{trial_result}...}}")
        results.append(trial_result)

```

```

total_instances = sum(result["instances_tested"] for result in results)
if total_instances == 0:
    print("RESULT: INCONCLUSIVE no_valid_instances")
    exit(1)

correlation_sum = sum(result["metric_value"] * result["instances_tested"] for result in results)
ratio_avg_sum = sum(result["metric_value"] * result["instances_tested"] for result in results)

mean_correlation = correlation_sum / total_instances
support_fraction = len([result for result in results if result["conjecture_holds"]]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_correlation} std=0 support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] and result["instances_tested"] > 0 for result in results):
    first_failing_seed = next((seed for seed, result in zip(seeds, results) if not result["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"correlation_threshold_not_met\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_537d776e.py", line 158, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_537d776e.py", line 110, in run_trial
    variables, clauses = tseitin_formula(f)
                          ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_537d776e.py", line 87, in tseitin_formula
    clause = [variables[2*j + 1], -variables[2*i + 1]]
              ~~~~~~^~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not complete the required calculations to verify the conjecture. | next: Re-run the test without errors to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14321
2	propose	ollama_remote	glm4:latest	0	0	12683
3	preregistration	ollama_remote	glm4:latest	0	0	9732
4	novelty	ollama_remote	glm4:latest	0	0	8197
5	novelty	ollama_remote	glm4:latest	0	0	9120
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15380
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9886
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8451
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16493
10	judge	ollama_remote	glm4:latest	0	0	11405

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 115669 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/c41170a41b57.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c41170a41b57.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/c41170a41b57.tar.gz` (if generated)